

NAME: Ayaan Shaikh

Mobile no : 8055733707

Internship : Data Analytics using Python

Submission date: 25/05/2026

### Task 91 : Create a data model using Linear Regression.

#### Objective

To predict a continuous numerical value (like sales, price, etc.) based on input variables and understand the relationship between dependent and independent variables.

```
# 91. Build a Linear Regression Model
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
df=pd.read_csv("iris.csv")
X=df[['sepal_length']]
y=df['sepal_width']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2)
model=LinearRegression()
model.fit(X_train,y_train)
predictions = model.predict(X_test)
predictions

array([3.01528877, 3.05437996, 3.05437996, 3.01528877, 3.0621982 ,
       3.18729003, 3.15601707, 3.07783468, 3.02310701, 3.03092524,
       3.05437996, 3.12474411, 3.0621982 , 3.08565292, 3.1012894 ,
       3.17947179, 3.12474411, 3.0621982 , 2.98401581, 3.10910764,
       3.01528877, 3.09347116, 3.12474411, 3.14819883, 2.99183405,
       3.17165355, 3.16383531, 3.11692588, 3.17947179, 3.01528877])
```

I have implemented a **Linear Regression model** using the Iris dataset to understand the relationship between two variables.

In my code, I first imported necessary libraries like Pandas and Scikit-learn. Then I loaded the dataset (iris.csv) and selected **sepal\_length** as the independent variable (X) and **sepal\_width** as the dependent variable (y). After that, I split the data into training and testing sets using train\_test\_split, where 80% of the data is used to train the model and 20% is used for testing.

Next, I created a **Linear Regression model** and trained it using the training data (model.fit). Once the model learned the relationship between sepal length and sepal width, I used it to make predictions on the test data (model.predict).

## Task 92 : Analyze data using Logistic Regression.

### Objective

To classify data into categories (e.g., Yes/No, 0/1) and estimate the probability of outcomes.

```
# 92. Perform Logistic Regression Analysis
from sklearn.linear_model import LogisticRegression
X=df[['sepal_length','sepal_width']]
y=df['species']
model=LogisticRegression()
model.fit(X,y)
prediction=model.predict([[5.7,2.8]])
print(prediction)

['versicolor']
```

I used **Logistic Regression** on the Iris dataset to classify the flower species. I selected **sepal length** and **sepal width** as input features and **species** as the target variable.

Then, I trained the model using the dataset. After training, I gave new input values [5.7, 2.8] to predict the species. The output shows which category the model predicts for the flower, demonstrating a basic classification task.

## Task 93 : Perform classification using Decision Trees or Random Forest.

### Objective:

To build models that classify data into different categories and improve accuracy using ensemble techniques (Random Forest).

```
# 93. Use Decision Trees / Random Forest for Classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
df=pd.read_csv("User_Data_Association.csv")
X=df[['Age','EstimatedSalary']]
y=df['Purchased']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
rf=RandomForestClassifier(random_state=42)
rf.fit(X_train,y_train)
# Prediction
new=[[30,100000]]
print("Prediction : ",rf.predict(new))

Prediction : [1]
```

I used a **Random Forest Classifier** on the dataset to predict whether a user will purchase or not. I selected **Age** and **Estimated Salary** as input features and **Purchased** as the target variable. Then, I split the data into training (80%) and testing (20%) sets using `train_test_split`.

After that, I created the Random Forest model and trained it using the training data. Once the model was trained, I provided a new input [30, 100000] (age and salary) to make a prediction. The output shows whether the model predicts the user will purchase or not, demonstrating a classification task using an ensemble learning method.

### Task 94 : Apply the Naive Bayes algorithm.

#### Objective

To perform fast and efficient classification based on probability, especially useful for text data like spam detection.

```
# 94. Apply Naive Bayes Algorithm
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
X=df[['Age','EstimatedSalary']]
y=df['Purchased']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
df['Gender'] = df['Gender'].map({'Male':1, 'Female':0})
nb=GaussianNB()
nb.fit(X_train,y_train)
new=[[30,100000]]
print("Prediction : ",nb.predict(new))

Prediction :  [0]
```

I used the **Naive Bayes (GaussianNB)** algorithm to predict whether a user will purchase or not. I selected **Age** and **Estimated Salary** as input features and **Purchased** as the target variable. I also converted the **Gender** column into numeric form (Male = 1, Female = 0) for preprocessing. Then, I split the dataset into training (80%) and testing (20%) sets.

After that, I trained the Naive Bayes model using the training data. Finally, I gave a new input [30, 100000] to predict the result. The output shows whether the model predicts the user will purchase or not, demonstrating a probability-based classification method.

## Task 95: Prepare data for KNN algorithm.

### Objective

To preprocess and structure data (normalization, scaling) so that KNN can classify data points based on similarity.

```
# 95. KNN (K-Nearest Neighbors)
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
df=pd.read_csv("iris.csv")
X=df[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']]
y = df['species']
le = LabelEncoder()
y = le.fit_transform(y)
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
new = [[5.1, 3.5, 1.4, 0.2]]
new = scaler.transform(new)
pred = knn.predict(new)
pred

C:\Users\MyLove\AppData\Roaming\Python\Python311\site-packages\sklearn\utils\validation.py:100: UserWarning:
StandardScaler was fitted with feature names
warnings.warn(
array([0])
```

I used the **K-Nearest Neighbors (KNN)** algorithm on the Iris dataset to classify flower species. I selected all four features (**sepal length, sepal width, petal length, petal width**) as inputs and **species** as the target variable. Since the target was categorical, I converted it into numeric form using Label Encoding.

Then, I split the data into training and testing sets. After that, I applied **StandardScaler** to normalize the data, which is important for KNN because it is distance-based. I trained the KNN model with k=3 neighbors using the training data.

Finally, I provided a new data point [5.1, 3.5, 1.4, 0.2], scaled it, and predicted its class. The output (pred) shows the predicted species label, demonstrating a similarity-based classification method.

## Task 96 : Identify clusters in K-Means clustering.

### Objective

To group similar data points into clusters and discover hidden patterns in unlabeled data.

```
# 96. Identify Clusters using K-Means
import pandas as pd
from sklearn.cluster import KMeans
df=pd.read_csv("iris.csv")
X = df[['sepal_length','sepal_width','petal_length','petal_width']]
kmeans=KMeans(n_clusters=3)
kmeans.fit(X)
df['Cluster']=kmeans.labels_
df.head()
```

C:\Users\MyLove\AppData\Roaming\Python\Python311\site-packages\sklearn\cluster\\_kmeans.py:105: UserWarning: Using a loop to construct data chunks with MKL, when there are less chunks than available threads. You may want to increase the number of chunks with n\_jobs.
warnings.warn(

|   | sepal_length | sepal_width | petal_length | petal_width | species | Cluster |
|---|--------------|-------------|--------------|-------------|---------|---------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | setosa  | 1       |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | setosa  | 1       |
| 2 | 4.7          | 3.2         | 1.3          | 0.2         | setosa  | 1       |
| 3 | 4.6          | 3.1         | 1.5          | 0.2         | setosa  | 1       |
| 4 | 5.0          | 3.6         | 1.4          | 0.2         | setosa  | 1       |

I used the **K-Means Clustering** algorithm on the Iris dataset to group the data into clusters.

I selected four features (**sepal length, sepal width, petal length, petal width**) as input variables. Then, I created a K-Means model with **3 clusters** and trained it using the dataset.

After training, the model assigned each data point to a cluster based on similarity. I stored these cluster labels in a new column called "**Cluster**" in the dataset.

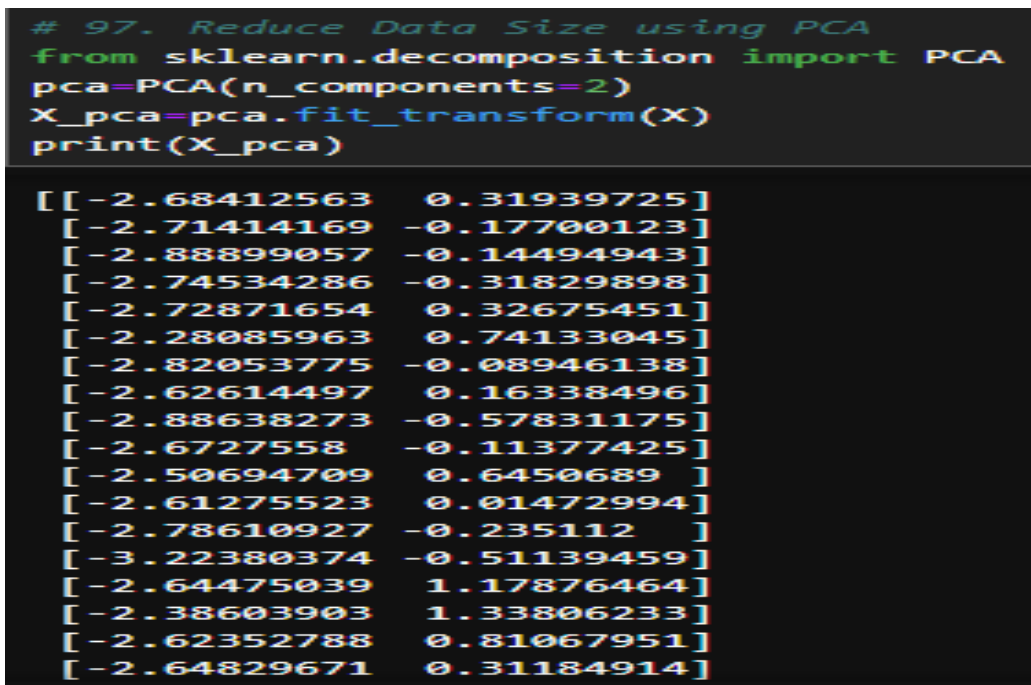
The `df.head()` shows the first few rows with their assigned clusters, demonstrating how unsupervised learning groups similar data points together.

## Task 97 : Reduce the size (dimensions) of data using PCA (Principal Component Analysis).

### Objective

To simplify large datasets while retaining important information, making analysis faster and more efficient.

```
# 97. Reduce Data Size using PCA
from sklearn.decomposition import PCA
pca=PCA(n_components=2)
X_pca=pca.fit_transform(X)
print(X_pca)
```



|             |             |
|-------------|-------------|
| -2.68412563 | 0.31939725  |
| -2.71414169 | -0.17700123 |
| -2.88899057 | -0.14494943 |
| -2.74534286 | -0.31829898 |
| -2.72871654 | 0.32675451  |
| -2.28085963 | 0.74133045  |
| -2.82053775 | -0.08946138 |
| -2.62614497 | 0.16338496  |
| -2.88638273 | -0.57831175 |
| -2.6727558  | -0.11377425 |
| -2.50694709 | 0.6450689   |
| -2.61275523 | 0.01472994  |
| -2.78610927 | -0.235112   |
| -3.22380374 | -0.51139459 |
| -2.64475039 | 1.17876464  |
| -2.38603903 | 1.33806233  |
| -2.62352788 | 0.81067951  |
| -2.64829671 | 0.31184914  |

I used Principal Component Analysis (PCA) on the dataset to reduce its dimensionality. I selected the feature set X and applied PCA with 2 components, which means the original data is compressed into two principal components.

Then, I fitted the PCA model and transformed the data using `fit_transform()`, which reduces the dataset while preserving most of the important information. The output `X_pca` contains the new transformed values in a lower-dimensional space.

This demonstrates a dimensionality reduction technique used to simplify data visualization and improve model efficiency.

## **Task 98 : Understand Basic Concepts of Neural Networks .**

### **Objective**

To understand how artificial neural networks work, including neurons, layers, weights, and activation functions.

### **1. What is a Neural Network**

- A machine learning model inspired by the human brain
- Used for classification and prediction tasks

### **2. Basic Structure**

- **Input Layer** → receives data (features)
- **Hidden Layer(s)** → processes data
- **Output Layer** → gives final result

### **3. Neuron (Node)**

- Basic unit of a neural network
- Each neuron takes input, applies weight, bias, and activation function

### **4. Weights and Bias**

- **Weights:** importance of each input
- **Bias:** adjusts output to improve accuracy

### **5. Activation Functions**

- Decide output of a neuron
- Common ones:
- Sigmoid (0 to 1)
- ReLU (removes negative values)
- Softmax (multi-class classification)

## Task 100 : Analyze data using Support Vector Machine.

### Objective

To classify data by finding the optimal boundary (hyperplane) that separates different classes with maximum margin.

```
# 100.Perform Analysis using SVM (Support Vector Machine)
from sklearn.svm import SVC
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
df=pd.read_csv("User_Data_Association.csv")

X=df[['Age','EstimatedSalary']]
y=df['Purchased']
#df['Gender'] = df['Gender'].map({'Male':1, 'Female':0})
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2)
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)
model=SVC(kernel='linear')
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
new=[[30,118000]]
print("Prediction using SVM : ",model.predict(new))

Prediction using SVM :  [1]
```

I used the Support Vector Machine (SVM) algorithm to predict whether a user will purchase a product or not. I selected Age and Estimated Salary as input features and Purchased as the target variable.

First, I split the dataset into training and testing sets. Then, I applied StandardScaler to normalize the features, which is important for SVM because it works based on distance and margins. After that, I trained an SVM model with a linear kernel using the training data.

Once the model was trained, I used it to predict results on test data and also gave a new input [30, 118000] to check the prediction. The output shows whether the model classifies the user as likely to purchase or not, demonstrating a margin-based classification method.



I visualized multi-dimensional data by creating a 3D scatter plot using Matplotlib. I loaded the iris dataset and plotted sepal length, sepal width, and petal length on the x, y, and z axes respectively.

This approach allowed me to represent three variables in a single visualization, making it easier to analyze relationships and observe how the variables interact with each other.

By using a 3D view, I was able to identify patterns, clusters, and possible correlations in the data more effectively than with a simple 2D chart, which helps in gaining deeper insights and improving data analysis.